

# 1. Visão Geral e Arquitetura do Sistema

## 1.1. A Escolha Arquitetural

A solução adota uma arquitetura cliente-servidor em três camadas: um Single Page Application (SPA) como camada de apresentação, que consome uma API RESTful (camada de aplicação e de regras de negócio), a qual é a única responsável por acessar o banco de dados relacional PostgreSQL (camada de dados). As requisições seguem de cima para baixo (front-end, API e banco de dados) e as respostas fazem o caminho inverso.

A escolha por cliente-servidor decorre da existência de dois papéis distintos, executados em processos independentes que se comunicam por requisições HTTP: o cliente, que roda no navegador do usuário, apresenta as telas, coleta a entrada e solicita os dados; e o servidor, que aguarda as requisições, valida, aplica as regras, grava e responde. Essa separação independe de onde cada parte é executada: na execução local ambos rodam na mesma máquina, mas, por se comunicarem apenas por HTTP, podem ser hospedados em máquinas distintas sem alteração de código. O servidor é a fonte única da verdade: permissões por perfil, fluxo de status, controle de concorrência e validação de dados são decididos nele, e a validação feita na interface serve apenas para melhorar a experiência de uso. A API também não mantém sessão do cliente: cada requisição carrega o próprio token de autenticação, o que mantém o servidor sem estado. A divisão em três camadas separa as responsabilidades e permite que cada parte evolua de forma independente. Interface e API são entregues como projetos distintos, de modo que a mesma API poderia atender outro cliente, como um aplicativo móvel, sem alterações. A segurança fica concentrada no servidor: o navegador nunca acessa o banco de dados, e o banco sequer é exposto fora da rede interna quando a solução é executada em contêineres.

O front-end é um SPA porque o sistema exige uma interface interativa, com formulários, listagens filtradas conforme o usuário digita e um dashboard com gráficos. Nesse modelo o servidor entrega um único documento HTML e a interface é montada no navegador, onde um roteador no lado do cliente alterna as telas sem recarregar a página, enquanto os dados trafegam em JSON por requisições à API.

A API é um monólito modular, e não um conjunto de microsserviços: um único processo, organizado em módulos por área funcional, o que reduz a complexidade operacional para o escopo do projeto sem impedir a separação futura. Na execução com Docker, cada camada roda em um contêiner próprio (front-end, API e banco de dados), refletindo a separação física descrita acima.

## 1.2. Estratégia de Comunicação

A comunicação entre o front-end e a API ocorre por requisições HTTP, com troca de dados em JSON, seguindo o estilo REST: cada recurso (solicitações, comentários, categorias) tem uma URL própria e as operações são expressas pelos verbos HTTP, com códigos de status que indicam o resultado (por exemplo, 400 para validação, 403 para falta de permissão e 409 para conflito de estado). Como o front-end e a API são executados em origens distintas, o acesso é controlado por CORS: a API libera apenas as origens configuradas, nunca

todas. Para evitar tráfego desnecessário, as listagens usam ETag e revalidação, e as atualizações periódicas, como as dos comentários e do dashboard, são feitas por consulta repetida (polling).

Entre a API e o banco de dados a comunicação não usa HTTP: a API abre conexões diretas com o PostgreSQL pelo protocolo nativo sobre TCP, por meio de um pool de conexões, e o acesso é intermediado pelo Prisma, que gera consultas tipadas e parametrizadas. A escolha de HTTP com JSON é o padrão de mercado para aplicações web: é suportada nativamente pelos navegadores, tem amplo ferramental de teste e depuração e não exige nada além do que o navegador já oferece. O contexto da aplicação, composto por operações de cadastro, consulta e atualização sem necessidade de recebimento de eventos em tempo real, é bem atendido pelo modelo de requisição e resposta, o que dispensa soluções mais complexas, como WebSocket. O protocolo nativo do PostgreSQL, por sua vez, é o meio padrão de acesso a esse banco.

### 1.3. Estratégia de Autenticação Global

A autenticação é baseada em tokens JWT (JSON Web Token) e é transversal a todo o sistema: ao informar usuário e senha, o cliente recebe da API um token assinado, de validade de uma hora, que contém a identidade e o perfil do usuário e que é enviado em cada requisição seguinte. O servidor não mantém sessão em memória nem em banco: toda a informação necessária para autenticar e autorizar uma requisição está no próprio token, o que mantém a API sem estado e simplifica a comunicação entre o SPA e o servidor.

Do lado do servidor, a segurança é aplicada de forma global e com negação por padrão: todas as rotas exigem um token válido, exceto o login, que é explicitamente público. Uma segunda verificação, baseada no perfil do usuário (Solicitante ou Atendente), restringe as operações exclusivas de cada perfil, e as regras mais específicas, como a propriedade da solicitação, são decididas na camada de serviço a partir da identidade presente no token, nunca de dados enviados pelo cliente. As senhas são armazenadas apenas como hash (bcrypt) e a mensagem de erro do login é a mesma para usuário inexistente e senha incorreta, o que evita revelar quais usuários existem.

A sessão é mantida no navegador, que guarda o token e o anexa às requisições. Ela é encerrada quando o usuário sai, caso em que o cliente descarta o token, ou quando o token expira: nesse caso a API responde com erro de autenticação e a interface limpa a sessão e redireciona para a tela de login. Optou-se por autenticação stateless por ser a alternativa mais simples e adequada a um SPA que consome uma API REST em outra origem: dispensa um armazenamento de sessões no servidor e evita a necessidade de cookies, com sua proteção contra CSRF. Em contrapartida, o token não pode ser revogado antes de expirar, risco que é mitigado pela validade curta de uma hora.

## 2. Modelagem e Persistência de Dados

### 2.1. Escolha do Banco de Dados (Justificativa Técnica)

Foi escolhido o PostgreSQL, um sistema de gerenciamento de banco de dados relacional, na versão 16.

A escolha se apoia em três fatores. O PostgreSQL é de código aberto e gratuito, o que elimina custos de licenciamento e permite executá-lo localmente sem restrições, inclusive em um contêiner Docker. É também um dos bancos de dados mais utilizados no mercado, com documentação extensa, comunidade ativa e ampla disponibilidade de ferramentas e bibliotecas, o que facilita a manutenção e a evolução do sistema. Por fim, a experiência prévia com bancos de dados relacionais favorece a produtividade dentro do prazo do projeto e reduz o risco técnico.

O modelo relacional, por sua vez, é adequado ao domínio do sistema: usuários abrem solicitações, que pertencem a categorias e possuem um histórico de transições e uma conversa associada. O banco garante a integridade desses vínculos por meio de chaves estrangeiras, restrições de unicidade e tipos enumerados, e oferece transações, usadas nas operações que precisam ser atômicas, como registrar uma solicitação junto com seu histórico ou impedir que dois atendentes assumam o mesmo chamado.

### 2.2. Estratégia de Modelagem

A modelagem divide os dados em entidades que correspondem aos conceitos do negócio, seguindo a normalização: cada informação é guardada em um único lugar e as demais tabelas a referenciam por chave estrangeira. Os casos de uso resultaram em quatro entidades principais (`usuarios`, `categorias`, `solicitacoes` e `historico_solicitacoes`) e em duas entidades de apoio à comunicação no chamado (`comentarios` e `comentarios_revisoes`).

A tabela `usuarios` concentra a autenticação e a autorização, com o perfil (`Solicitante` ou `Atendente`) definindo os privilégios. A tabela `categorias` representa os setores como dados, e não como um tipo fixo no código, e o campo `ativa` permite retirar uma opção de uso sem corromper as solicitações já registradas. A tabela `solicitacoes` é o núcleo transacional, ligando a demanda a quem a abriu e à categoria escolhida. A tabela `historico_solicitacoes` registra cada transição de status, com o estado anterior e o novo, o autor e o instante. Por fim, `comentarios` guarda a conversa do chamado e `comentarios_revisoes` guarda o texto anterior de cada edição.

Duas decisões orientam o modelo. A primeira é não armazenar o que pode ser derivado: o atendente responsável, a data de conclusão e a data da última atualização não são colunas, e sim obtidos do histórico, que passa a ser a única fonte dessas informações e elimina o risco de divergência entre campos. A segunda é usar tipos nativos do banco para os domínios fechados (`status` e `perfil` como enumerados, datas com fuso horário) e definir explicitamente o comportamento das chaves estrangeiras, com exclusão em cascata apenas onde faz sentido (`histórico` e `comentários` acompanham a solicitação excluída) e restrição nas demais. A estrutura é versionada por migrações geradas a partir de um único esquema, que é a fonte

da verdade do banco; o detalhamento de tabelas, colunas e índices está no dicionário de dados.

## 2.3. Resolução de Complexidades

A primeira complexidade é a relação muitos-para-muitos entre solicitações e usuários. Uma solicitação passa por vários usuários ao longo de sua vida (o solicitante que a abre e os atendentes que a movimentam) e um mesmo atendente altera várias solicitações. Como o modelo relacional não representa essa ligação diretamente, a tabela `historico_solicitacoes` funciona como entidade associativa: ela transforma a relação em duas relações um-para-muitos (uma solicitação possui vários registros de histórico e um usuário é autor de vários registros) e ainda carrega os atributos próprios da ligação, o status anterior, o status novo e o instante da operação. O resultado é uma trilha de auditoria completa, sem duplicar dados nas tabelas principais.

A segunda é definir quem é o responsável por um chamado sem criar uma coluna que possa divergir do histórico: o responsável é quem fez a primeira transição para Em Atendimento, informação que já está registrada. Para impedir que dois atendentes assumam o mesmo chamado ao mesmo tempo, a condição é colocada na própria escrita (uma atualização que só altera a linha se o status ainda for o esperado), dentro de uma transação junto com o registro do histórico. De duas tentativas simultâneas, apenas uma tem efeito e a outra recebe um conflito, de modo que a garantia vem do banco e não depende de uma verificação prévia na aplicação.

A terceira é preservar o passado. Categorias fora de uso são marcadas como inativas, e não excluídas, e as chaves estrangeiras com restrição impedem a exclusão de usuários e categorias em uso, de modo que nenhum registro fique órfão.

Por fim, a auditoria dos comentários é resolvida sem perder informação nem expô-la: a exclusão é lógica (a linha permanece, com a data e o autor da exclusão, e deixa de aparecer na API) e cada edição gera uma revisão com o texto anterior, gravada na mesma transação da alteração. Essa trilha é interna e serve à consulta direta no banco. A única exceção é a exclusão de uma solicitação inteira, que remove também seus comentários e revisões.

## 3. O Backend (Regras de Negócio e API)

### 3.1. Tecnologia Principal (Justificativa Técnica)

Para o back-end foi escolhido o framework NestJS, em TypeScript, sobre Node.js. O NestJS fornece nativamente uma arquitetura modular em camadas, com módulos, controllers e providers (services) ligados por injeção de dependência, enquanto frameworks minimalistas, como o Express, deixam a organização da aplicação a cargo do desenvolvedor. Ele utiliza o próprio Express como servidor HTTP, o que mantém a compatibilidade com o ecossistema, mas impõe uma estrutura padronizada.

A escolha favorece a manutenção e a produtividade: o padrão de organização reduz decisões de arquitetura, torna o código previsível para quem o lê e facilita os testes, já que as dependências são injetadas. O uso de TypeScript acrescenta tipagem estática, que reduz

erros e documenta os dados trocados entre as camadas, e o framework oferece módulos oficiais para necessidades recorrentes, como configuração e autenticação com JWT.

## 3.2. Organização Estrutural e Padrões

O código do servidor é organizado em módulos por área funcional (auth, categorias, solicitacoes, comentarios e dashboard), em um módulo global de acesso ao banco de dados (prisma) e em uma pasta common, que reúne os elementos transversais. Dentro de cada módulo as responsabilidades são divididas em camadas: o controller recebe a requisição e delega, o service concentra as regras de negócio e os DTOs definem e validam os dados de entrada. O acesso ao banco de dados é feito exclusivamente na camada de serviço, por meio do PrismaService, que encapsula o cliente do Prisma e a conexão com o PostgreSQL. Os controllers não acessam o banco.

Os trechos a seguir, extraídos do código, mostram cada camada.

### Módulo: agrupa o controller e o service de uma área (comentarios.module.ts)

```
@Module({
  controllers: [ComentariosController],
  providers: [ComentariosService],
})
export class ComentariosModule {}
```

### Controller: recebe a requisição, extrai os dados e delega ao service (solicitacoes.controller.ts)

```
@Patch('/:codigo/status')
@Roles(PerfilUsuario.ATENDENTE)
alterarStatus(
  @Param('codigo', ParseIdPipe) codigo: number,
  @Body() dto: AlterarStatusDto,
  @UsuarioAtual() usuario: UsuarioAutenticado,
) {
  return this.solicitacoesService.alterarStatus(codigo, dto, usuario);
}
```

### DTO: define e valida a entrada da API (criar-solicitacao.dto.ts)

```
export class CriarSolicitacaoDto {
  @TextoObrigatorio(LIMITES.titulo)
  titulo!: string;

  @TextoObrigatorio(LIMITES.descricao)
  descricao!: string;

  @IdInteiro()
  categoriaId!: number;
}
```

### Service: regras de negócio. Escrita condicional que impede dois atendentes de assumirem o mesmo chamado (assuncao.ts, trecho simplificado)

```
const { count } = await tx.solicitacao.updateMany({
  where: { codigo, status: atual },
  data: { status: novo },
});
if (count === 0) {
  throw new ConflictException(
```

```

        'Este chamado já foi assumido por outro atendente; atualize a tela',
    );
}

```

### Acesso a dados: o PrismaService é injetado no service e é a única porta para o banco

```

constructor(private readonly prisma: PrismaService) {}

// consulta feita pelo service (comentarios.service.ts)
this.prisma.comentario.findMany({
  where: { solicitacaoCodigo: codigo, excluidoEm: null },
  orderBy: { id: 'asc' },
})

```

### Transversal: decorator criado no projeto para obter o usuário autenticado (usuario-atual.decorator.ts)

```

export const UsuarioAtual = createParamDecorator(
  (_data: unknown, context: ExecutionContext): UsuarioAutenticado =>
    context.switchToHttp().getRequest().usuario,
);

```

Os padrões de projeto utilizados são, em sua maioria, os oferecidos pelo próprio NestJS. Não foram implementados padrões próprios, como Factory ou Strategy.

| Padrão                                        | Onde aparece                                                                                                                 |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Injeção de dependência (inversão de controle) | O contêiner do NestJS injeta os services e o PrismaService pelo construtor.                                                  |
| Módulo                                        | Um módulo por área funcional.                                                                                                |
| Decorator                                     | @Controller, @Roles, @Public, o @UsuarioAtual criado no projeto e os decoradores de validação compostos com applyDecorators. |
| DTO (Data Transfer Object)                    | Classes que definem e validam os dados que entram pela API.                                                                  |
| Singleton                                     | Os providers do NestJS são instâncias únicas por padrão, como o PrismaService.                                               |
| Camada de serviço (Service Layer)             | Controller fino e regras de negócio concentradas no service.                                                                 |
| Máquina de estados simples                    | A tabela de transições PROXIMO_STATUS define o fluxo Aberto, Em Atendimento e Concluído.                                     |
| Concorrência otimista                         | A condição vai na própria escrita (UPDATE com WHERE no status esperado), dentro de uma transação.                            |

## 3.3. Detalhamento de Ferramentas Auxiliares

Além do framework, o back-end utiliza bibliotecas menores, cada uma escolhida para resolver um problema específico. As tabelas a seguir resumem o problema, a ferramenta e a forma como ela foi usada.

### Em tempo de execução

| Ferramenta                              | Problema que resolve                           | Como foi usada                                                                                                                                                                                                             |
|-----------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| class-validator e class-transformer     | Validar e converter os dados de entrada.       | Decoradores nos DTOs, aplicados por um ValidationPipe global que converte tipos e recusa campos desconhecidos. Decoradores próprios, construídos sobre eles, padronizam os limites de tamanho e rejeitam o caractere nulo. |
| bcrypt                                  | Guardar as senhas sem expô-las em texto.       | Armazenamento apenas do hash (com salt) e comparação no login.                                                                                                                                                             |
| @nestjs/jwt                             | Autenticar sem manter sessão no servidor.      | Gera o token no login, com validade de uma hora, e verifica sua assinatura em cada requisição, em um guard global.                                                                                                         |
| @nestjs/config                          | Configurar a aplicação sem segredos no código. | Lê variáveis de ambiente, como o segredo do JWT, que é obrigatório para a API iniciar.                                                                                                                                     |
| Prisma (cliente, adaptador e driver pg) | Acessar o banco com segurança e produtividade. | Consultas tipadas e parametrizadas, transações e pool de conexões com o PostgreSQL. No dashboard, SQL puro parametrizado para as agregações.                                                                               |
| dotenv                                  | Carregar variáveis de ambiente fora do NestJS. | Usado na configuração do Prisma e nos scripts de seed.                                                                                                                                                                     |

### Em desenvolvimento e qualidade

| Ferramenta                          | Problema que resolve                             | Como foi usada                                                                                              |
|-------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Prisma CLI (migrations)             | Evoluir o esquema do banco de forma controlada.  | Migrações versionadas geradas a partir de um único esquema e aplicadas automaticamente na subida em Docker. |
| tsx                                 | Executar scripts TypeScript sem compilar.        | Execução dos scripts de seed (dados iniciais e de demonstração).                                            |
| Vitest, Supertest e @nestjs/testing | Verificar o comportamento de forma automatizada. | Testes unitários dos services e DTOs e testes de ponta a ponta com requisições HTTP reais contra o banco.   |
| oxlint e Prettier                   | Manter o código padronizado.                     | Análise estática (lint) e formatação do código.                                                             |
| TypeScript e Nest CLI               | Tipagem estática e compilação.                   | Compilação do projeto e execução em modo de observação durante o desenvolvimento.                           |

### 3.4. Estratégia de Testes

A qualidade do back-end é verificada por testes automatizados em dois níveis, ambos executados com o Vitest: testes unitários, rápidos e isolados do banco de dados, e testes de ponta a ponta (e2e), que exercitam a API real, com requisições HTTP e um PostgreSQL de verdade. Os níveis se complementam: o primeiro localiza falhas de lógica com precisão e o segundo garante que as camadas funcionam juntas. No total são 292 testes unitários, executados com `npm test`, e 148 testes de ponta a ponta, executados com `npm run test:e2e`.

**Testes unitários.** Cada classe é testada isoladamente. O service é instanciado diretamente, sem o contêiner do NestJS, recebendo um PrismaService simulado, feito com funções de simulação (mocks) do Vitest. Isso permite controlar o que o banco responde em cada cenário e verificar o que o service tentou gravar. Os testes seguem o padrão preparar, executar e verificar, os mocks são reiniciados antes de cada caso e os nomes descrevem o comportamento esperado. Eles cobrem as regras de negócio (permissões por perfil, fluxo de status, trava de concorrência, chamado concluído somente leitura), os DTOs e os decoradores de validação, que são testados com o mesmo ValidationPipe da aplicação e com valores-limite (por exemplo, 255 e 256 caracteres), e as funções puras, como o cálculo de períodos do dashboard.

**Exemplo de teste unitário: o atendente que perde a corrida pela assunção recebe conflito e nada é gravado (comentarios.service.spec.ts)**

```
it('perdeu a corrida para outro atendente: 409 e nada é gravado', async () => {
  findChamado
    .mockResolvedValueOnce(aberto)
    .mockResolvedValueOnce({
      ...emAtendimento,
      historico: [{ usuario: { id: 8, nome: 'Atendente Dois' } }],
    });
  updateMany.mockResolvedValue({ count: 0 });

  await expect(
    service.criar(10, { texto: 'Oi' }, atendente),
  ).rejects.toBeInstanceOf(ConflictException);
  expect(criarComentario).not.toHaveBeenCalled();
  expect(criarHistorico).not.toHaveBeenCalled();
});
```

**Exemplo de teste de validação com valores-limite (comentarios.dto.spec.ts)**

```
it('aceita exatamente o limite e recusa um a mais', async () => {
  await expect(corpo('a'.repeat(LIMITES.comentario))).resolves.toBeDefined();
  await expect(corpo('a'.repeat(LIMITES.comentario + 1))).rejects.toBeInstanceOf(
    BadRequestException,
  );
});
```

**Testes de ponta a ponta.** Os testes e2e sobem a aplicação completa em memória, a partir do módulo raiz e com a mesma configuração de produção (validação global e CORS), e disparam requisições HTTP com o Supertest contra um PostgreSQL real, com as migrações e o seed aplicados. Cada suíte cria os próprios dados, identificados por um marcador único, e os remove ao final, de modo que o banco continua como estava. Além da resposta HTTP, os testes consultam o banco diretamente para confirmar o efeito da operação, como o histórico gravado, a exclusão lógica e as revisões dos comentários. Os cenários de concorrência são simulados com requisições simultâneas, verificando que apenas uma

vence. Esses testes cobrem a matriz de respostas (400, 401, 403, 404 e 409), os limites de validação, a revalidação com ETag, a paginação e os filtros, o dashboard e a comunicação no chamado.

### **Preparação: a aplicação real é montada com a configuração de produção (test/app.e2e-spec.ts)**

```
beforeAll(async () => {
  const moduleFixture: TestingModule = await Test.createTestingModule({
    imports: [AppModule],
  }).compile();

  app = moduleFixture.createNestApplication();
  process.env.CORS_ORIGIN = 'http://localhost:5173';
  configurarApp(app);
  await app.init();
});
```

### **Exemplo de teste e2e: duas assunções simultâneas, uma vence e o banco confirma um único responsável (test/app.e2e-spec.ts)**

```
it('duas assunções simultâneas: uma passa (200) e a outra perde (409)', async () => {
  const codigo = await criar('Dono corrida');

  const respostas = await Promise.all([
    mudarStatus(atendente, codigo, 'EM_ATENDIMENTO'),
    mudarStatus(atendente2, codigo, 'EM_ATENDIMENTO'),
  ]);
  const statuses = respostas.map((r) => r.status).sort();

  expect(statuses).toEqual([200, 409]);
  const assuncoes = await app.get(PrismaService).historicoSolicitacao.findMany({
    where: { solicitacaoCodigo: codigo, statusNovo: 'EM_ATENDIMENTO' },
  });
  expect(assuncoes).toHaveLength(1); // uma só linha: não há dois responsáveis
});
```

O front-end ainda não possui testes automatizados, o que está registrado nas limitações da solução (seção 6.1).

## **4. O Frontend (Interface e Interação do Usuário)**

### **4.1. Tecnologia Principal (Justificativa Técnica)**

Para o front-end foi escolhida a biblioteca React, em JavaScript, com o Vite como ferramenta de build e servidor de desenvolvimento. O React constrói a interface a partir de componentes que se atualizam conforme o estado, o que se adequa ao que o sistema exige: formulários com validação imediata, uma listagem cujos filtros e busca reagem enquanto o usuário digita e um dashboard com gráficos que se redesenham a cada mudança de período ou de setor.

### **Componente: peça reutilizável da interface, que recebe dados por parâmetro (Shared.jsx)**

```
export function StatusBadge({ status }) {
  const s = STATUS[status]
  return (
    <span className="badge" style={{ '--c': s.color }}>
```

```

    <i className="dot" />
    {s.label}
  </span>
)
}

```

O ecossistema do React fornece as peças de que o projeto precisa: o React Router, para a navegação entre telas; o TanStack Query, para o cache das consultas; o Recharts, para os gráficos do dashboard (pizza, barras e área); o Lucide, para os ícones; e o Motion, para as transições. O Vite oferece um servidor de desenvolvimento com atualização instantânea e gera um pacote estático, que é a forma de entrega do SPA descrita na seção 1.

## 4.2. Organização dos Componentes e Navegação

O código-fonte é organizado por responsabilidade, em pastas dentro de src: pages, com uma tela por arquivo (Login, Dashboard, Solicitações, Detalhe da solicitação e Nova solicitação); components, com as partes reutilizáveis, como o Layout com o menu lateral, o Modal, a página de erro, a conversa do chamado e os campos do formulário de solicitação; hooks, com a lógica reutilizável, como a busca de categorias, a conversa do chamado e o controle de ações em andamento; services, com o acesso à API; e utils, com a validação dos campos. O estado global da aplicação, que reúne o usuário autenticado, as notificações e o erro global, fica em um contexto (context.jsx) disponível a todas as telas.

A navegação usa o React Router, com as rotas declaradas de forma aninhada. Todas as telas passam por um componente de proteção, que só exibe o conteúdo quando há usuário autenticado (caso contrário, mostra o login), e compartilham o Layout, que traz o menu lateral, a trilha de navegação e as transições entre telas. As rotas são o dashboard (/), a listagem de solicitações (/solicitacoes), o detalhe (/solicitacoes/:id) e a nova solicitação (/nova, restrita ao solicitante), com uma tela de página não encontrada para as demais. Os filtros da listagem podem ser recebidos pela URL, de modo que o clique em um indicador do dashboard abre a listagem já filtrada, e a saída da tela de nova solicitação com dados preenchidos pede confirmação.

**Rotas aninhadas: todas as telas passam pelo componente de proteção e compartilham o Layout (App.jsx)**

```

const router = createBrowserRouter([
  {
    element: <Shell />,
    errorElement: <ErrorPage variant="generic" fullscreen />,
    children: [
      {
        element: <Protected />,
        children: [
          {
            element: <Layout />,
            children: [
              { index: true, element: <Dashboard /> },
              { path: 'solicitacoes', element: <Requests /> },
              { path: 'solicitacoes/:id', element: <RequestDetail /> },
              { path: 'nova', element: <RequesterOnly><NewRequest /></RequesterOnly> },
              { path: '*', element: <ErrorPage variant="notfound" /> },
            ],
          },
        ],
      },
    ],
  },
],

```

```

    ],
  },
])

```

**Proteção de rotas: sem usuário autenticado o conteúdo não é exibido; o atendente não abre solicitações (App.jsx, trecho)**

```

function Protected() {
  const { user, authError, dismissAuthError } = useApp()
  if (user) return <Outlet />
  if (authError) return <ErrorPage variant="unauthorized" fullscreen
  onAction={dismissAuthError} actionLabel="Fazer login" />
  return <Login />
}

function RequesterOnly({ children }) {
  const { isAgent } = useApp()
  return isAgent ? <Navigate to="/" replace /> : children
}

```

**Confirmação de saída: a tela de nova solicitação bloqueia a navegação enquanto há dados preenchidos (NewRequest.jsx)**

```

const dirty = f.title.trim() !== '' || f.description.trim() !== ''

const blocker = useBlocker(
  ({ currentLocation, nextLocation }) =>
    dirty && !allowLeave.current && currentLocation.pathname !== nextLocation.pathname,
)

```

## 4.3. Consumo da API

Toda a comunicação com a API passa por uma única camada. O módulo http.js centraliza a URL base (definida em tempo de build), o envio do token em cada requisição, a conversão do corpo em JSON e o tratamento de erros. Sobre ele, o módulo api.js expõe uma função para cada operação do contrato e converte os dados da API para o formato usado pelas telas, de modo que mudanças no contrato ficam concentradas nesses dois arquivos.

**Camada única de acesso: token em todas as requisições e tratamento padronizado de erros (http.js, trecho)**

```

const headers = {}
if (body !== undefined) headers['Content-Type'] = 'application/json'
const token = session.get()?.accessToken
if (auth && token) headers.Authorization = `Bearer ${token}`

res = await fetch(`${BASE_URL}${path}${qs ? `?${qs}` : ''}`, { method, headers, body:
... })

if (!res.ok) {
  if (res.status === 401 && auth) {
    session.clear()
    onUnauthorized()
  }
  throw new ApiError(res.status, friendlyMessage(res.status) || parseMessage(data, `Erro
${res.status}`))
}

```

### Adaptação ao contrato: uma função por operação, com os filtros enviados ao servidor e a resposta convertida (api.js)

```
export async function listRequests({ statuses, categoryId, assignee, q, from, to, page = 1, size = 20 } = {}) {
  const data = await request('/solicitacoes', {
    query: {
      status: statuses,
      categoriaId: categoryId,
      atendente: assignee,
      q: q?.trim(),
      dataInicio: from,
      dataFim: to,
      pagina: page,
      tamanho: size,
    },
  })
  return {
    items: data.itens.map(toListItem),
    total: data.total,
    page: data.pagina,
    size: data.tamanho,
    totalPages: data.totalPaginas,
  }
}
```

O tratamento das respostas é padronizado. A mensagem devolvida pela API é lida e exibida ao usuário, os erros de servidor e de tamanho de conteúdo recebem textos amigáveis em português e a falta de conexão é informada. Uma resposta 401 em qualquer tela encerra a sessão, limpa o cache e leva o usuário ao login. A interface diferencia o tipo de falha: as validações aparecem junto ao campo, o sucesso de uma ação aparece em uma notificação, a falha de uma ação aparece em um modal e a falha ao carregar uma tela (por exemplo, uma solicitação inexistente ou sem acesso) aparece em uma página de erro. Os botões ficam bloqueados enquanto a ação está em andamento, o que evita envios duplicados. Os dados do dashboard, das categorias e dos comentários são mantidos em cache e atualizados periodicamente, enquanto a listagem consulta o servidor a cada mudança de filtro, com um pequeno atraso na busca por texto (debounce de 300 ms).

### Envio duplicado: o hook executa uma ação por vez e bloqueia cliques repetidos (useAction.js)

```
export function useAction() {
  const locked = useRef(false)
  const [busy, setBusy] = useState(false)

  const run = useCallback(async (fn) => {
    if (locked.current) return
    locked.current = true
    setBusy(true)
    try {
      return await fn()
    } finally {
      locked.current = false
      setBusy(false)
    }
  }, [])

  return [run, busy]
}
```

## Cache e atualização periódica do dashboard com o TanStack Query (Dashboard.jsx, trecho)

```
const { data, isPending, isPlaceholderData, dataUpdatedAt, error } = useQuery({
  queryKey: ['dashboard', filters],
  queryFn: () => api.getDashboard(filters),
  placeholderData: keepPreviousData,
  enabled: !invalidRange,
  refetchOnWindowFocus: true,
  refetchInterval: REFRESH_MS,
  refetchIntervalInBackground: false,
})
```

O perfil do usuário, recebido no login, determina o que a interface mostra. O Solicitante vê apenas as próprias solicitações, tem acesso à tela de nova solicitação e pode editar ou excluir os seus chamados que ainda estão abertos. O Atendente vê todas as solicitações, dispõe dos filtros por atendente (meus, todos e sem atendente), do escopo geral ou pessoal no dashboard e do painel para alterar o status, que fica bloqueado quando outro atendente é o responsável pelo chamado; ao tentar abrir a tela de nova solicitação, é redirecionado. Essa diferenciação é uma conveniência de uso: as regras de permissão são aplicadas e validadas pela API, que continua recusando qualquer operação não permitida.

## Interface por perfil: o menu e o painel de status mudam conforme o usuário (Layout.jsx e RequestDetail.jsx, trechos)

```
{NAV.filter((n) => !n.requesterOnly || !isAgent).map(({ to, end, icon: Icon, label }) =>
(
  <NavLink key={to} to={to} end={end} title={label}>...</NavLink>
))}

const statusBlock = !isAgent
  ? 'Apenas atendentes podem alterar o status'
  : r.status === 'EM_ATENDIMENTO' && r.assigneeId && r.assigneeId !== user.id
    ? `Somente ${r.assignee} pode alterar o status deste chamado`
    : ''
```

# 5. Infraestrutura e Setup

## 5.1. Ferramentas de Ambiente/Deploy

Para padronizar o ambiente e facilitar a execução por quem for avaliar o projeto, a solução foi empacotada com Docker e Docker Compose. O objetivo é que toda a aplicação (banco de dados, API e interface) suba com um único comando, sem exigir a instalação de Node.js nem de PostgreSQL e sem configuração manual.

A escolha se justifica pela reprodutibilidade: cada componente roda em uma imagem com as dependências e as versões definidas, de modo que o resultado não depende da máquina de quem executa. O Docker também automatiza as etapas que normalmente são manuais (criar o banco, aplicar as migrações, popular os dados iniciais e configurar as variáveis de ambiente), o que reduz o risco de falha na avaliação.

A solução é composta por três serviços. O banco de dados usa a imagem oficial do PostgreSQL 16, com volume para persistência e verificação de saúde. A API é construída em duas etapas (compilação e execução) e, ao iniciar, um script aplica as migrações, cria as

categorias e os usuários de teste e, opcionalmente, carrega chamados de demonstração apenas quando ainda não existe nenhum. A interface é compilada pelo Vite e servida pelo nginx, que redireciona as rotas do SPA para o index.html e comprime as respostas. A ordem de inicialização é garantida por verificações de saúde: a API só sobe depois do banco, e a interface depois da API.

Todas as variáveis têm valores padrão no próprio arquivo do Compose, portanto não é preciso criar um arquivo .env; ele é opcional, para trocar portas, segredos ou desligar os dados de demonstração. A interface é construída a partir do repositório do front-end no GitHub ou de uma pasta local, e o endereço da API é gravado no momento do build, porque quem a chama é o navegador do usuário. Basta executar `docker compose up --build` e acessar `http://localhost:5173`, e o passo a passo, com os usuários de teste, está no README do projeto.

#### Inicialização da API: prepara o banco e sobe o servidor (docker-entrypoint.sh)

```
prisma migrate deploy
prisma db seed

if [ "${SEED_DEMO:-false}" = "true" ]; then
  tsx prisma/seed-demo.ts --se-vazio
fi

exec node dist/main
```

#### Orquestração: ordem de subida e valores padrão (docker-compose.yml, trecho)

```
api:
  build: .
  depends_on:
    db:
      condition: service_healthy
  environment:
    JWT_SECRET: ${JWT_SECRET:-segredo- apenas-para-avaliacao-local}
    SEED_DEMO: ${SEED_DEMO:-true}

front:
  build:
    context: ${FRONT_CONTEXT:-https://github.com/Lord1nho/psel-BIT-front.git#main}
  args:
    VITE_API_URL: http://localhost:${API_PORT:-3000}
```

## 6. Análise Crítica e Próximos Passos

### 6.1. Limitações da Solução Implementada

**Sessão e autenticação.** O token JWT não pode ser revogado: o logout apenas o descarta no navegador, e ele continua válido até expirar, em uma hora. Também não há renovação do token, ele fica no armazenamento local do navegador, onde um ataque de XSS poderia lê-lo, e não existe limite de tentativas de login.

**Transporte.** A execução entregue usa HTTP sem criptografia. O HTTPS depende da implantação.

**Usuários e categorias.** Existem apenas pelo seed: não há cadastro, edição, troca ou recuperação de senha.

**Atendimento.** Quem assume o chamado é o único que pode concluí-lo, e não há como transferi-lo ou liberá-lo se o responsável estiver ausente. Um chamado concluído não pode ser reaberto.

**Comunicação no chamado.** Não há tempo real: a tela consulta novos comentários a cada 15 segundos. Também não há notificações nem anexos, e edições ou exclusões feitas pela outra pessoa só aparecem ao recarregar a conversa.

**Auditoria.** A trilha de edições e exclusões dos comentários é interna, consultável apenas no banco, e é apagada junto com o chamado quando um solicitante exclui uma solicitação aberta.

**Qualidade.** O back-end tem testes unitários e de ponta a ponta, mas o front-end não tem testes automatizados e não há pipeline de integração contínua. O desempenho sob carga foi verificado apenas de forma pontual.

**Regras duplicadas.** Os limites de validação são repetidos manualmente no front-end e no back-end, sem um contrato compartilhado, o que exige manter os dois em sincronia.

**Acoplamento.** Os services acessam o Prisma diretamente, sem uma camada de repositório, o que os torna dependentes dessa biblioteca.

## 6.2. Evolução para Produção

Para publicar o sistema para milhares de funcionários em um ambiente corporativo, os principais pontos de evolução seriam os seguintes.

**Segurança.** HTTPS em todo o tráfego, por meio de um proxy reverso; renovação e revogação de tokens (por exemplo, com uma lista de bloqueio em um cache) ou cookies httpOnly com proteção contra CSRF; limite de tentativas de login; autenticação corporativa por SSO, LDAP ou Active Directory, em vez de usuários locais; segredos em um cofre; e atenção à LGPD no tratamento dos dados pessoais.

**Entrega contínua.** Pipeline de CI/CD com lint, testes e build das imagens versionadas em um registro, com ambientes de homologação e produção, deploy automatizado e migrações executadas pelo pipeline.

**Dados.** PostgreSQL gerenciado, com backup e restauração a qualquer instante, réplicas de leitura para relatórios, pool de conexões dedicado e política de retenção e arquivamento.

**Escala e desempenho.** O desenho sem estado da API facilita executar várias instâncias atrás de um balanceador, com orquestração (como Kubernetes). Um cache, como o Redis, poderia atender o dashboard e as consultas agregadas quando a medição indicar necessidade, e uma CDN serviria os arquivos estáticos do front-end, com compressão também na API.

**Comunicação assíncrona.** Mensageria (filas) para enviar notificações por e-mail e push sem prender a requisição, e WebSocket ou SSE para entregar comentários em tempo real.

**Observabilidade e qualidade.** Logs estruturados, métricas, rastreamento e alertas; testes automatizados de interface, de ponta a ponta, e testes de carga.

## 6.3. Possíveis Melhorias de Requisitos

**Notificações.** Avisos por e-mail e na própria tela quando o status muda ou chega um novo comentário.

**Prioridade e SLA.** Definir prioridade e prazos de atendimento por prioridade ou categoria, com alertas de vencimento (a interface já reserva os campos).

**Anexos.** Permitir enviar arquivos e imagens nos chamados e nos comentários.

**Gestão do atendimento.** Transferência e liberação de chamados, reabertura de chamados concluídos, equipes por setor e distribuição automática dos chamados.

**Administração.** Telas para gerenciar usuários e categorias, e para trocar e recuperar a senha.

**Comunicação.** Comentários internos entre atendentes, invisíveis para o solicitante, e indicador de mensagens não lidas.

**Indicadores e relatórios.** Tempo médio de atendimento, indicadores por atendente e exportação de relatórios em CSV ou PDF.

**Satisfação.** Pesquisa de satisfação do solicitante ao concluir o chamado.